

```

1 import numpy as np
2 import tensorflow as tf
3 import matplotlib.pyplot as plt
4 from tensorflow.keras import layers, models
5 (x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
6 x_train = x_train.astype("float32")/255.0
7 x_test = x_test.astype("float32")/255.0
8 x_train = x_train.reshape((len(x_train), 784))
9 x_test = x_test.reshape((len(x_test), 784))

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 0s 0us/step

```

1 input_dim=784
2 encoding_dim=32
3 input_layer=layers.Input(shape=(input_dim,))
4 encoder=layers.Dense(encoding_dim,activation="relu")(input_layer)
5 decoder=layers.Dense(input_dim,activation="sigmoid")(encoder)
6 autoencoder=models.Model(input_layer,decoder)
7
8 encoder=models.Model(input_layer,encoder)
9
10 autoencoder.compile(optimizer="adam",loss="binary_crossentropy")

```

1 Start coding or [generate](#) with AI.

```
1 autoencoder.fit(x_train,x_train,epochs=20,batch_size=256,shuffle=True,validation_data=(x_test,x_test))
```

```

Epoch 1/20
235/235 ————— 3s 7ms/step - loss: 0.2768 - val_loss: 0.1894
Epoch 2/20
235/235 ————— 1s 3ms/step - loss: 0.1703 - val_loss: 0.1521
Epoch 3/20
235/235 ————— 1s 3ms/step - loss: 0.1433 - val_loss: 0.1331
Epoch 4/20
235/235 ————— 1s 3ms/step - loss: 0.1283 - val_loss: 0.1212
Epoch 5/20
235/235 ————— 1s 3ms/step - loss: 0.1184 - val_loss: 0.1129
Epoch 6/20
235/235 ————— 1s 4ms/step - loss: 0.1113 - val_loss: 0.1069
Epoch 7/20
235/235 ————— 1s 5ms/step - loss: 0.1060 - val_loss: 0.1025
Epoch 8/20
235/235 ————— 1s 3ms/step - loss: 0.1020 - val_loss: 0.0989
Epoch 9/20
235/235 ————— 1s 3ms/step - loss: 0.0991 - val_loss: 0.0966
Epoch 10/20
235/235 ————— 1s 3ms/step - loss: 0.0973 - val_loss: 0.0951
Epoch 11/20
235/235 ————— 1s 3ms/step - loss: 0.0960 - val_loss: 0.0942
Epoch 12/20
235/235 ————— 1s 3ms/step - loss: 0.0953 - val_loss: 0.0936
Epoch 13/20
235/235 ————— 1s 3ms/step - loss: 0.0948 - val_loss: 0.0932
Epoch 14/20
235/235 ————— 1s 3ms/step - loss: 0.0944 - val_loss: 0.0930
Epoch 15/20
235/235 ————— 1s 3ms/step - loss: 0.0941 - val_loss: 0.0926
Epoch 16/20
235/235 ————— 1s 3ms/step - loss: 0.0939 - val_loss: 0.0925
Epoch 17/20
235/235 ————— 1s 3ms/step - loss: 0.0938 - val_loss: 0.0924
Epoch 18/20
235/235 ————— 1s 3ms/step - loss: 0.0936 - val_loss: 0.0922
Epoch 19/20
235/235 ————— 1s 3ms/step - loss: 0.0935 - val_loss: 0.0922
Epoch 20/20
235/235 ————— 1s 3ms/step - loss: 0.0934 - val_loss: 0.0921
<keras.src.callbacks.history.History at 0x7bdf231a41d0>

```

```

1 encoded_imgs=encoder.predict(x_test)
2 reconstructed_imgs=autoencoder.predict(x_test)

```

```

313/313 ————— 1s 2ms/step
313/313 ————— 1s 2ms/step

```

```

1 encoder=layers.Dense(128,activation="relu")(input_layer)
2 encoder=layers.Dense(64,activation="relu")(encoder)
3 encoder=layers.Dense(32,activation="relu")(encoder)
4

```

```
5 decoder=layers.Dense(64,activation="relu")(encoder)
6 decoder=layers.Dense(128,activation="relu")(decoder)
7 decoder=layers.Dense(784,activation="sigmoid")(decoder)
```

1 Start coding or [generate](#) with AI.

```
1 from tensorflow.keras import regularizers
2 (x_train,_),(x_test,_)=tf.keras.datasets.mnist.load_data()
3 x_train=x_train.astype("float32")/255.0
4 x_test=x_test.astype("float32")/255.0
5 x_train=x_train.reshape((len(x_train),784))
6 x_test=x_test.reshape((len(x_test),784))
7
```

```
1
2 input_dim=784
3 encoding_dim=32
4 input_layer=layers.Input(shape=(input_dim,))
5 encoder=layers.Dense(encoding_dim,activation="relu",activity_regularizer=regularizers.l1(1e-5))(input_layer)
6 decoder=layers.Dense(input_dim,activation="sigmoid")(encoder)
7 autoencoder=models.Model(input_layer,decoder)
8
9 encoder=models.Model(input_layer,encoder)
10
```

```
1
2 autoencoder.compile(optimizer="nadam",loss="binary_crossentropy")
3 autoencoder.fit(x_train,x_train,epochs=20,batch_size=256,shuffle=True,validation_data=(x_test,x_test))
```

```
Epoch 1/20
235/235 ————— 4s 13ms/step - loss: 0.4013 - val_loss: 0.2942
Epoch 2/20
235/235 ————— 1s 3ms/step - loss: 0.2590 - val_loss: 0.2353
Epoch 3/20
235/235 ————— 1s 3ms/step - loss: 0.2269 - val_loss: 0.2183
Epoch 4/20
235/235 ————— 1s 3ms/step - loss: 0.2138 - val_loss: 0.2080
Epoch 5/20
235/235 ————— 1s 3ms/step - loss: 0.2050 - val_loss: 0.2003
Epoch 6/20
235/235 ————— 1s 3ms/step - loss: 0.1981 - val_loss: 0.1940
Epoch 7/20
235/235 ————— 1s 3ms/step - loss: 0.1925 - val_loss: 0.1894
Epoch 8/20
235/235 ————— 1s 3ms/step - loss: 0.1883 - val_loss: 0.1856
Epoch 9/20
235/235 ————— 1s 3ms/step - loss: 0.1850 - val_loss: 0.1827
Epoch 10/20
235/235 ————— 1s 3ms/step - loss: 0.1824 - val_loss: 0.1805
Epoch 11/20
235/235 ————— 1s 3ms/step - loss: 0.1801 - val_loss: 0.1782
Epoch 12/20
235/235 ————— 1s 3ms/step - loss: 0.1781 - val_loss: 0.1764
Epoch 13/20
235/235 ————— 1s 3ms/step - loss: 0.1764 - val_loss: 0.1750
Epoch 14/20
235/235 ————— 1s 4ms/step - loss: 0.1748 - val_loss: 0.1731
Epoch 15/20
235/235 ————— 1s 4ms/step - loss: 0.1733 - val_loss: 0.1718
Epoch 16/20
235/235 ————— 1s 4ms/step - loss: 0.1720 - val_loss: 0.1703
Epoch 17/20
235/235 ————— 1s 3ms/step - loss: 0.1708 - val_loss: 0.1693
Epoch 18/20
235/235 ————— 1s 3ms/step - loss: 0.1697 - val_loss: 0.1679
Epoch 19/20
235/235 ————— 1s 3ms/step - loss: 0.1686 - val_loss: 0.1670
Epoch 20/20
235/235 ————— 1s 3ms/step - loss: 0.1676 - val_loss: 0.1661
<keras.src.callbacks.history.History at 0x7bdf2337ca10>
```

```
1 encoded_imgs=encoder.predict(x_test)
2 reconstructed_imgs=autoencoder.predict(x_test)
```

```
313/313 ————— 1s 2ms/step
313/313 ————— 1s 2ms/step
```

```
1 from tensorflow.keras import activations
2 import tensorflow.keras.backend as K
3 from tensorflow.keras import regularizers
4 (x_train,y_train),(x_test,y_test)=tf.keras.datasets.mnist.load_data()
```

```
5 x_train=x_train.astype("float32")/255.0
6 x_test=x_test.astype("float32")/255.0
7 x_train=x_train.reshape((len(x_train),784))
8 x_test=x_test.reshape((len(x_test), 784))
```

```
1 input_dim=784
2 encoding_dim=64
3 lam=1e-4
4
5 inputs=layers.Input(shape=(input_dim,))
6 encoded=layers.Dense(encoding_dim,activation="sigmoid")(inputs)
7 decoded=layers.Dense(input_dim,activation="sigmoid")(encoded)
8 autoencoder=models.Model(inputs,decoded)
9
10 # Define the encoder model specifically for this autoencoder
11 encoder = models.Model(inputs, encoded)
12
13 def contractive_loss(y_true,y_pred):
14     recon_loss=tf.keras.losses.binary_crossentropy(y_true,y_pred)
15     W=autoencoder.layers[1].kernel
16     h=autoencoder.layers[1].output
17     dh=h*(1-h)
18     contractive=lam*K.sum(K.square(W),axis=0)*K.sum(K.square(dh),axis=1)
19     return recon_loss+contractive
```

```
1 autoencoder.compile(optimizer="adam",loss="binary_crossentropy")
2 autoencoder.fit(x_train,x_train,epochs=50,batch_size=256,shuffle=True,validation_data=(x_test,x_test))
```

```
235/235 ————— 1s 3ms/step - loss: 0.0833 - val_loss: 0.0822
Epoch 23/50
235/235 ————— 1s 3ms/step - loss: 0.0826 - val_loss: 0.0816
Epoch 24/50
235/235 ————— 1s 3ms/step - loss: 0.0820 - val_loss: 0.0811
Epoch 25/50
235/235 ————— 1s 4ms/step - loss: 0.0814 - val_loss: 0.0806
Epoch 26/50
235/235 ————— 1s 5ms/step - loss: 0.0810 - val_loss: 0.0802
Epoch 27/50
235/235 ————— 1s 3ms/step - loss: 0.0806 - val_loss: 0.0798
Epoch 28/50
235/235 ————— 1s 3ms/step - loss: 0.0802 - val_loss: 0.0794
Epoch 29/50
```

<keras.src.callbacks.history.History at 0x7bdf18128d70>

```
1 # Extract encoded features for training and testing
2 x_train_encoded = encoder.predict(x_train)
3 x_test_encoded = encoder.predict(x_test)
```

1875/1875 ————— 2s 1ms/step
313/313 ————— 1s 2ms/step

```
1 # Define a simple classifier on top of the encoded features
2 classifier_input = layers.Input(shape=(encoding_dim,))
3 classifier_output = layers.Dense(10, activation='softmax')(classifier_input)
4 classifier = models.Model(classifier_input, classifier_output)
5
6 classifier.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
7
8 # Train the classifier
9 print("\nTraining the classifier...")
10 classifier.fit(x_train_encoded, y_train, epochs=50, batch_size=256, shuffle=True, validation_data=(x_test_encoded, y_test))
```

Training the classifier...

Epoch 1/10
235/235 ————— 3s 8ms/step - accuracy: 0.2836 - loss: 2.2183 - val_accuracy: 0.5329 - val_loss: 1.8761
Epoch 2/10
235/235 ————— 1s 3ms/step - accuracy: 0.6500 - loss: 1.6566 - val_accuracy: 0.7442 - val_loss: 1.4393
Epoch 3/10
235/235 ————— 1s 3ms/step - accuracy: 0.7625 - loss: 1.3055 - val_accuracy: 0.7963 - val_loss: 1.1533
Epoch 4/10
235/235 ————— 1s 3ms/step - accuracy: 0.7959 - loss: 1.0778 - val_accuracy: 0.8140 - val_loss: 0.9716
Epoch 5/10
235/235 ————— 1s 3ms/step - accuracy: 0.8116 - loss: 0.9278 - val_accuracy: 0.8255 - val_loss: 0.8464
Epoch 6/10
235/235 ————— 1s 3ms/step - accuracy: 0.8216 - loss: 0.8248 - val_accuracy: 0.8326 - val_loss: 0.7594
Epoch 7/10
235/235 ————— 1s 3ms/step - accuracy: 0.8285 - loss: 0.7504 - val_accuracy: 0.8380 - val_loss: 0.6954
Epoch 8/10
235/235 ————— 1s 3ms/step - accuracy: 0.8343 - loss: 0.6948 - val_accuracy: 0.8435 - val_loss: 0.6476
Epoch 9/10
235/235 ————— 1s 3ms/step - accuracy: 0.8385 - loss: 0.6519 - val_accuracy: 0.8492 - val_loss: 0.6092
Epoch 10/10
235/235 ————— 1s 3ms/step - accuracy: 0.8433 - loss: 0.6175 - val_accuracy: 0.8515 - val_loss: 0.5787
<keras.src.callbacks.history.History at 0x7bdf18101580>

```
1 # Evaluate the classifier
2 loss, accuracy = classifier.evaluate(x_test_encoded, y_test)
3 print(f"\nClassifier Test Accuracy: {accuracy*100:.2f}%")
```

313/313 ————— 1s 3ms/step - accuracy: 0.8515 - loss: 0.5787

Classifier Test Accuracy: 85.15%

1 Start coding or [generate](#) with AI.

```
1 encoded_imgs=encoder.predict(x_test)
2 reconstructed_imgs=autoencoder.predict(x_test)
```

313/313 ————— 1s 2ms/step
313/313 ————— 1s 2ms/step

1 Start coding or [generate](#) with AI.

